

Design and Implementation of an FPGA-Based Smart Elevator Controller with UART Telemetry and Real-Time Web Monitoring

Submitted towards the partial fulfillment for the requirements of

VIII Semester Major Project (22ECW451)

End-Term Evaluation

Submitted by

Name : Bhavesh Gurnani(2022UEC1410)

Name : Ojasvi Aggarwal (2022UEC1614)

Name : Mohd Anas Ansari (2022UEC1713)

Name : Saroj Kumar (2022UEC1785)

Under the Supervision of

Prof. Lava Bhargava



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
MALAVIYA NATIONAL INSTITUTE OF TECHNOLOGY JAIPUR

MAY 2026

CERTIFICATE

This is to certify that the VIII Semester Major Project (22ECW451) report entitled **“Design and Implementation of an FPGA-Based Smart Elevator Controller with UART Telemetry and Real-Time Web Monitoring”** submitted by Bhavesh Gurnani(2022UEC1410) ; Ojasvi Aggarwal(2022UEC1614) ; Mohd. Anas Ansari(2022UEC1713) ; Saroj Kumar(2022UEC1785) ; is a project work carried out under my supervision.

(Signature)

Place : MNIT Jaipur

Supervisor Name : Prof. Lava Bhargava

Date :

Dept. of Electronics and Comm. Engineering

ACKNOWLEDGMENTS

We express our sincere gratitude to our mentor **Prof. Lava Bhargava**, of Dept. of Electronics and Communication Engineering, MNIT Jaipur who has guided us throughout the VIII Semester Major Project (22ECW451) work on “**Design and Implementation of an FPGA-Based Smart Elevator Controller with UART Telemetry and Real-Time Web Monitoring**”. His/Her valuable insights, feedback and support have been instrumental in shaping the project work content and quality of the report. We would like to thank **Prof. M. M. Sharma**, Head, Department of Electronics and Communication Engineering, for his support and providing a wonderful atmosphere to do the research work. We would like to thank Director **Prof. N. P. Padhy** for providing the needed resources and facilities in the institute.

Name : Bhavesh Gurnani(Roll Number :2022UEC1410)

Name : Ojasvi Aggarwal (Roll Number : 2022UEC1614)

Name : Mohd. Anas Ansari(Roll Number :2022UEC1713)

Name : Saroj Kumar (Roll Number : 2022UEC1785)

ABSTRACT

This project presents the design and hardware implementation of a smart elevator control system using FPGA technology and Verilog Hardware Description Language (HDL). The system has been developed on the Digilent Basys3 FPGA development board using the Xilinx Vivado Design Suite. The aim of the project is to build a real-time elevator controller capable of handling multiple floor requests efficiently while also providing live system monitoring through a computer-based interface.

The elevator controller operates using a Finite State Machine (FSM) model in which different operating conditions such as idle state, upward movement, downward movement, door operation, and emergency handling are represented as separate states. The controller accepts requests generated both from inside the elevator cabin and from external floor buttons. Based on the active requests and the current direction of movement, the system determines the next floor to be served in an optimized manner to minimize unnecessary movement and improve response efficiency.

The complete elevator logic is implemented in Verilog HDL using modular design methodology. Different modules are responsible for request processing, floor decision logic, movement control, display handling, and UART communication. The system also includes door status indication, seven-segment display output, LED-based status monitoring, and an emergency stop mechanism. During emergency conditions, the controller safely moves the elevator to the nearest floor, opens the door, and suspends further operation until a reset signal is applied.

To extend the functionality of the hardware system, UART-based serial communication is integrated into the FPGA design. Elevator status data is continuously transmitted from the FPGA board to a computer through the onboard USB-UART interface. A Python-based middleware application receives and interprets the serial data and forwards it to a WebSocket server. A browser-based dashboard developed using HTML, CSS, and JavaScript provides real-time graphical visualization of elevator activity, including floor movement, request indication, direction status, door operation, and emergency alerts.

The proposed system demonstrates the effective use of FPGA technology for real-time control applications. The project combines digital hardware design, embedded communication, and live software-based monitoring into a unified system. Experimental testing on FPGA hardware verified the correct operation of request scheduling, floor traversal, UART communication, emergency handling, and live monitoring features. The project highlights the potential of FPGA-based solutions in modern automation and intelligent control systems.

CONTENTS

Chapters	Details	Page. Nos.
Chapter 1.	Introduction	6
1.1	Introduction to elevator Control System	
1.2	Need for smart Elevator Controllers	
1.3	Problem Statement	
1.4	Motivation of the project	
1.5	Objectives of the project	
1.6	Scope of the project	
1.7	Features of proposed system	
1.8	Applications of FPGA based Elevator System	
1.9	Advantages of FPGA design	
1.10	Organisation of report	
Chapter 2.	Literature Review	13
2.1	Introduction to Digital System Design	
2.2	introduction to FPGA	
2.3	Fpga Architecture	
2.4	Working Principle of FPGA	
2.5	Introduction to Basys 3 FPGA Board	
2.6	FSM	
2.7	Types of FSM	
2.8	FSM used in this Project	
2.9	Elevator Scheduling Concepts	
2.10	Introduction to Hardware Description Language	
2.11	Introduction to Verilog	
2.12	Clock and timing concept	
2.13	UART communication protocol	
2.14	UART in this project	
2.15	Websockets	
2.16	Websockets in this project	
2.17	Python for Embedded Communication	
2.18	Frontend Technologies used	
1.19	Real Time Monitoring System	
2.20	Literature Review	
Chapter 3.	System Analysis and Requirement	20
3.1	Existing System analysis	
3.2	Proposed System Analysis	
3.3	Functional Requirements	
3.4	Non functional Requirements	
3.5	Hardware Requirements	
3.6	Software Requirements	
3.7	Feasibility Study	
3.8	Requirement Specification Table	

Chapter 4.	System Design and Architecture	27
4.1	Overall System Architecture	
4.2	High level block Diagram	
4.3	Data flow architecture	
4.4	Elevator Operational flow	
4.5	FSM design of elevator controller	
4.6	Request Scheduling Algorithm	
4.7	Emergency Handling Design	
4.8	Door Control System Design	
4.9	Clock Divider design	
4.10	UART Telemetry Architecture	
4.11	UART FSM Design	
4.12	Seven Segment Display Design	
4.13	LED indication system	
4.14	Web Dashboard Architecture	
4.15	Database and data storage Considerations	
Chapter 5.	Implementation and Software Interface	35
5.1	Introduction to module wise design	
5.2	Top level module	
5.3	Elevator core module	
5.4	Floor logic module	
5.5	Request logic module	
5.6	UART transmission Module	
5.7	XDC constraint file	
5.8	RTL schematic explanation	
5.9	Resource Utilisation	
5.10	Time analysis	
5.11	FPGA Hardware implementation	
5.12	Python UART Bridge	
5.13	UART testing utility	
5.14	Websocket server design	
5.15	JSON state structure	
5.16	Web Bases Monitoring dashboard	
5.17	Frontend Component design	
5.18	Javascript Logic	
5.19	Real time communication flow	
5.20	Advantage of monitoring system	
Chapter 6	Simulation and Experimental Results	46
6.1	Introduction to verification	
6.2	Simulation Testing	
6.3	Functional Testing	
6.4	Emergency Condition Testing	
6.5	UART communication testing	
6.6	Websocket communication testing	
6.7	Display Testing	

6.8	Hardware Testing on FPGA	
6.9	Test cases and results	
6.10	waveform analysis	
6.11	performance analysis	
6.12	Error analysis	
6.13	Predictive Maintenance feature	
6.14	Experimental Results and discussions	
Chapter 7	Conclusion and Future works	57
7.1	Conclusion	
7.2	Futureworks	
	References	62

Chapter 1. Introduction

1.1 Introduction to Elevator Control Systems

Elevators have become one of the most important transportation systems in modern buildings. They are used to move people and materials efficiently between different floors in residential apartments, hospitals, offices, shopping malls, airports, hotels, and industrial facilities. As the number of high-rise buildings continues to increase, the requirement for safe, reliable, and intelligent elevator systems has also grown significantly.

The earliest elevator systems were manually operated and depended heavily on mechanical arrangements and human operators. Later, relay-based elevator systems were introduced, where electrical relays and contactors controlled motor operation, floor selection, and door movement. Although these systems improved automation, they required large wiring arrangements, occupied more physical space, consumed higher power, and were difficult to maintain.

With the development of embedded electronics, microcontroller-based elevator systems became more common. Microcontrollers allowed programmable control logic and reduced hardware complexity. These systems improved flexibility and reduced cost; however, they still faced challenges in handling multiple operations simultaneously because most tasks were executed sequentially through software instructions.

To overcome such limitations, FPGA-based elevator systems provide a more advanced solution. A Field Programmable Gate Array (FPGA) allows digital hardware circuits to be configured according to application requirements. Using Verilog HDL, elevator control logic can be implemented directly in hardware, enabling high-speed operation and real-time processing. FPGA technology supports parallel execution of multiple tasks, making it highly suitable for automation and control systems.

Automation plays an important role in modern infrastructure. Intelligent elevator systems improve passenger comfort, reduce waiting time, optimize floor scheduling, and increase safety during emergency situations. The addition of communication and monitoring technologies further enhances system performance and reliability.

1.2 Need for Smart Elevator Controllers

Modern buildings require elevator systems that can respond quickly and operate efficiently under varying passenger traffic conditions. Traditional elevator systems

often suffer from slower response and inefficient movement when handling multiple requests simultaneously. Smart elevator controllers help overcome these limitations by using intelligent scheduling and automated control mechanisms.

Efficient request handling is one of the most important requirements in elevator systems. Requests may be generated from inside the elevator cabin as well as from different floors. A smart controller must process all these requests accurately without missing any request signal.

Another important requirement is the reduction of passenger waiting time. By optimizing elevator movement and serving requests in a planned order, unnecessary travel can be minimized and system efficiency can be improved.

Modern elevator systems must also support concurrent request processing. Multiple users may request the elevator at the same time, and the controller should be capable of handling all requests while maintaining smooth operation and proper direction control.

Safety is another major concern in elevator systems. Emergency handling mechanisms are necessary to protect passengers during abnormal conditions such as emergency stops or power failures. Real-time monitoring also improves maintenance and operational supervision by providing continuous information about elevator status and movement.

Reliability and fault tolerance are essential in automated systems because elevator failure can affect both safety and convenience. Therefore, modern elevator controllers require high-speed operation, dependable control logic, and accurate system monitoring.

1.3 Problem Statement

Conventional elevator systems face several technical and operational limitations, especially in buildings with large numbers of users. Relay-based systems require complicated wiring structures and occupy more hardware space, while software-oriented systems may suffer from slower execution during heavy request conditions.

One of the major issues in traditional elevator systems is inefficient sequential request processing. When multiple floor requests occur simultaneously, the elevator may perform unnecessary movement, increasing waiting time and reducing efficiency.

Another challenge is the absence of proper real-time monitoring and visualization support. Without live system monitoring, it becomes difficult to observe elevator activity, analyze faults, and perform debugging effectively.

Safety during emergency conditions is also a critical concern. During power interruptions or emergency situations, passengers may become trapped inside the elevator if proper protective mechanisms are not available.

In many software-controlled systems, operations are executed sequentially rather than in parallel, which can reduce response speed under complex conditions. Therefore, there is a requirement for a high-performance elevator controller capable of real-time operation, intelligent scheduling, safety management, and live monitoring.

1.4 Motivation of the Project

The main motivation behind this project is to study and implement a real-time embedded control system using FPGA technology and Verilog HDL. Elevator systems provide a practical example for understanding Finite State Machine (FSM) design, request scheduling algorithms, digital logic implementation, and hardware-software integration.

The project also helps in learning FPGA-based system development using the Xilinx Vivado Design Suite and the Digilent Basys3 development board. By implementing the elevator controller directly in hardware, important concepts such as synchronous design, clock division, state transitions, and modular digital design can be explored effectively.

Another motivation is to combine hardware control with software-based monitoring. The integration of UART communication, Python middleware, WebSocket communication, and browser visualization demonstrates the concept of hardware-software co-design for real-time monitoring applications.

The project also provides practical experience in developing automation systems that include communication interfaces, graphical monitoring dashboards, and emergency safety features.

1.5 Objectives of the Project

The main objectives of the proposed project are listed below:

- To design and implement a multi-floor elevator controller using FPGA technology.
- To develop elevator movement and scheduling logic using a Finite State Machine (FSM).
- To create modular Verilog HDL components compatible with FPGA implementation.
- To implement UART-based communication between the FPGA and computer system.

- To develop a real-time web-based monitoring interface for elevator visualization.
- To implement emergency stop and safety mechanisms for abnormal conditions.
- To provide LED and seven-segment display-based status indication.
- To study real-time embedded system design and digital automation techniques.

1.6 Scope of the Project

The project focuses on the development of a five-floor elevator control system implemented on an FPGA platform. The system supports cabin requests, hall UP requests, and hall DOWN requests while maintaining efficient scheduling and direction control.

The scope of the project includes FPGA-based hardware implementation using Verilog HDL, UART telemetry, WebSocket communication, and browser-based real-time monitoring. The project also includes animated elevator visualization, emergency stop handling, LED status indication, and seven-segment display support.

The system is designed to demonstrate real-time embedded control, hardware-software integration, and intelligent elevator scheduling using FPGA technology.

1.7 Features of the Proposed System

The proposed elevator controller includes several important features:

- Cabin floor request handling.
- Hall UP and Hall DOWN request processing.
- Dynamic floor scheduling based on elevator direction.
- Emergency stop operation for passenger safety.
- Automatic door opening and closing logic.
- UART-based serial communication for data transfer.
- Web-based real-time monitoring dashboard.
- Seven-segment display support for floor and status indication.
- LED-based operational status monitoring.

1.8 Applications of FPGA Based Elevator Systems

FPGA-based elevator systems can be used in several automation and embedded applications, including:

- Smart buildings and automated infrastructure.
- Industrial automation systems.
- Embedded monitoring and control systems.
- Real-time safety-critical applications.
- Transportation and mobility automation systems.
- Educational and research-based FPGA projects.

1.9 Advantages of FPGA-Based Design

FPGA-based systems provide multiple advantages over traditional control methods:

- Parallel processing capability for simultaneous operations.
- Predictable and deterministic timing behavior.
- High reliability in real-time applications.
- Flexible and reconfigurable hardware architecture.
- Low latency and fast system response.
- Hardware-level execution without operating system dependency.
- Better scalability for future modifications and enhancements.

1.10 Organization of the Report

This report is divided into multiple chapters that explain the design, implementation, testing, and analysis of the proposed FPGA-based elevator controller system.

Chapter 1 introduces the basic concepts of elevator control systems, project motivation, objectives, scope, features, applications, and advantages of FPGA-based implementation.

Chapter 2 presents the theoretical background and literature review related to FPGA technology, FSM concepts, Verilog HDL, UART communication, and real-time monitoring systems.

Chapter 3 explains the overall system architecture, hardware-software integration, elevator scheduling logic, UART telemetry structure, and module-level design.

Chapter 4 describes the implementation process, simulation environment, testing procedures, hardware verification, and performance analysis of the system.

Chapter 5 summarizes the project outcomes, conclusions, and possible future enhancements for the proposed system.

Chapter 2. Literature Review

2.1 Introduction to Digital System Design

Digital system design forms the basis of modern electronic and embedded systems. Digital circuits operate using binary logic and are generally categorized into combinational and sequential systems. In combinational circuits, outputs depend only on present inputs, whereas sequential systems also depend on previous states stored using memory elements such as registers and flip-flops. Counters, timing circuits, and synchronized clock signals are essential components of sequential digital hardware.

Synchronous digital systems rely on clock signals to coordinate hardware operations and state transitions. Important timing parameters such as setup time, hold time, synchronization, and metastability directly influence hardware reliability and performance. Hardware Description Languages (HDLs) are widely used for implementing digital systems because they simplify hardware modeling, scalability, and modularity. FPGA-based elevator controllers heavily utilize sequential logic, FSM structures, counters, and clock-driven control modules for real-time operation.

2.2 Introduction to FPGA

Field Programmable Gate Arrays (FPGAs) are programmable semiconductor devices capable of implementing customized hardware logic after manufacturing. Unlike software-based processors that execute instructions sequentially, FPGA systems implement hardware-level logic structures that can operate concurrently. This parallel processing capability allows multiple operations to execute simultaneously, making FPGA technology highly effective for real-time applications such as automation systems, robotics, communication systems, and intelligent elevator controllers [3].

FPGA technology evolved as a flexible alternative to fixed-function ASICs. While ASICs provide high performance for dedicated applications, they cannot be reprogrammed after fabrication. FPGAs overcome this limitation by allowing repeated hardware reconfiguration according to application requirements. Compared to traditional microcontroller-based systems, FPGA systems provide lower latency, deterministic timing behavior, scalability, and improved handling of simultaneous operations [3].

2.3 FPGA Architecture

FPGA architecture consists of configurable hardware resources connected through programmable routing structures. Major components include Configurable Logic

Blocks (CLBs), Lookup Tables (LUTs), flip-flops, routing matrices, DSP slices, Block RAMs, clock management resources, and input/output blocks.

Lookup Tables implement combinational logic functions, while flip-flops provide sequential storage for synchronous systems. Routing matrices connect hardware blocks together and allow flexible circuit implementation. DSP slices accelerate arithmetic operations, and Block RAM resources provide internal memory support for complex systems.

Clock management modules generate synchronized clock signals with varying frequencies required by different hardware subsystems. Input/output blocks interface the FPGA with external hardware such as LEDs, switches, displays, and communication interfaces. The parallel architecture of FPGAs makes them highly suitable for real-time embedded control systems [1].

2.4 Working Principle of FPGA

FPGA implementation begins with hardware modeling using an HDL such as Verilog. The HDL code is synthesized into logic gates and sequential elements using FPGA development tools. During implementation, placement and routing algorithms map the logic design onto the physical FPGA resources. The final configuration is converted into a bitstream file used to program the FPGA hardware [2].

Modern FPGA development environments automate synthesis, timing analysis, routing, implementation, and hardware programming. Once programmed, the FPGA behaves as a dedicated hardware circuit designed specifically for the target application.

2.5 Introduction to Basys3 FPGA Board

The proposed project uses the Digilent Basys3 FPGA development board, which is based on the Xilinx Artix-7 FPGA architecture [1]. The board includes several onboard peripherals such as slide switches, push buttons, LEDs, seven-segment displays, clock generators, USB programming support, and UART communication interfaces.

The Basys3 board is widely used for educational and embedded system development because it provides a complete FPGA experimentation platform without requiring additional hardware modules. Its integrated CP2104 USB-UART converter allows serial communication between the FPGA and a computer using the same USB cable used for programming [1].

2.6 Finite State Machines (FSM)

Finite State Machines (FSMs) are widely used in sequential digital systems because they provide organized state-based control mechanisms. FSMs transition between different states depending on inputs and clock signals. Since elevator systems naturally operate through state transitions, FSM architecture is highly suitable for elevator control implementation.

Typical elevator states include idle mode, upward movement, downward movement, door operation, and emergency handling. FSM-based design improves modularity, predictability, timing reliability, and debugging efficiency in FPGA systems [4].

2.7 Types of FSM

FSMs are commonly categorized into Moore and Mealy machines. In Moore FSMs, outputs depend only on the present state, while Mealy FSMs generate outputs based on both present states and inputs.

Moore architectures generally provide stable outputs and simplified timing behavior, whereas Mealy architectures often require fewer states and faster response. Elevator control systems commonly prefer Moore-style behavior because operational outputs such as direction control and door states are closely associated with stable system states [4].

2.8 FSM Used in This Project

The proposed elevator system uses an FSM-based architecture implemented entirely in Verilog HDL. Different states represent elevator conditions such as idle operation, upward movement, downward movement, door handling, and emergency control. Additional FSM logic is also used for UART data transmission sequencing.

The controller continuously monitors floor requests, movement direction, and emergency conditions. Based on active requests, the FSM determines movement direction and stopping conditions. FSM-based implementation was selected because it provides deterministic operation, scalability, modularity, and efficient hardware control [16].

2.9 Elevator Scheduling Concepts

Elevator scheduling algorithms play a major role in improving operational efficiency and reducing passenger waiting time. Intelligent elevator systems generally maintain movement in the current direction until all requests in that direction are completed before reversing.

Scheduling concepts include request queues, upward scheduling, downward scheduling, direction persistence logic, idle-state handling, and request prioritization. These techniques reduce unnecessary movement and improve floor traversal efficiency.

Several elevator controllers use scheduling methods inspired by the SCAN disk scheduling algorithm because it minimizes unnecessary reversals and optimizes request servicing [8].

2.10 Introduction to Hardware Description Languages

Hardware Description Languages (HDLs) are specialized languages used for modeling and implementing digital hardware systems. HDLs support structural modeling, behavioral modeling, and dataflow-based hardware description methodologies.

HDLs enable implementation of combinational circuits, FSMs, counters, communication interfaces, registers, and timing logic in modular and scalable forms. Verilog HDL is one of the most commonly used languages for FPGA system development because of its simplicity and strong hardware-oriented syntax [5].

2.11 Introduction to Verilog

Verilog HDL is widely used for FPGA and digital hardware implementation. It supports modular design through modules and ports, enabling reusable and scalable hardware architectures. Verilog includes constructs such as wires, registers, always blocks, case statements, blocking assignments, non-blocking assignments, parameters, and generated blocks.

Sequential systems are commonly implemented using clock-triggered always blocks, while combinational logic is implemented using continuous assignments or combinational always blocks. The IEEE 1364-2005 standard formally defines the syntax and operational semantics of Verilog HDL [5].

2.12 Clock and Timing Concepts

Clock management is a critical aspect of FPGA-based digital systems because all synchronous operations depend on stable timing signals. FPGA systems often use clock division circuits to generate slower clocks required for FSM operation, display multiplexing, and communication timing.

Timing concepts such as synchronization, setup time, hold time, frequency scaling, and metastability are important for stable hardware operation. Improper timing synchronization may lead to unpredictable state transitions and hardware instability.

Clock division is especially important in elevator systems because physical movement and display operations require slower timing than the default FPGA system clock frequency [2].

2.13 UART Communication Protocol

UART (Universal Asynchronous Receiver Transmitter) is one of the most commonly used serial communication protocols in embedded systems. UART enables asynchronous communication between hardware devices and computers using start bits, data bits, optional parity bits, and stop bits.

UART communication is widely adopted because of its simplicity, low hardware requirement, and compatibility with embedded devices and computer systems. Standard UART communication includes baud rate generation, serial framing, asynchronous synchronization, and byte-wise transmission [6].

2.14 UART in This Project

The proposed project integrates UART telemetry to continuously transmit elevator state information from the FPGA hardware to a computer system. The UART system uses a structured 4-byte frame containing current floor information, movement direction, request status, door activity, idle condition, and emergency state.

The UART transmitter operates at 9600 baud and uses asynchronous serial communication. Frame synchronization techniques are implemented using marker bits to ensure reliable data decoding on the software side. The onboard USB-UART interface available on the Basys3 board simplifies FPGA-to-PC communication without requiring external hardware [1].

2.15 WebSockets

WebSockets provide persistent duplex communication between software servers and web browsers. Unlike traditional HTTP communication, WebSockets maintain continuous bidirectional connections that support low-latency real-time data transfer [9].

WebSocket communication is widely used in industrial dashboards, IoT systems, monitoring applications, and real-time visualization systems because it allows live updates without repeated client requests.

2.16 WebSockets in This Project

The proposed system uses WebSocket communication to provide live browser-based elevator monitoring. A Python middleware application receives UART telemetry data from the FPGA and broadcasts the processed information to connected browser clients through a local WebSocket server.

This communication mechanism enables real-time updates of floor movement, request indicators, direction status, door activity, and emergency alerts. Browser-side WebSocket APIs support continuous communication and live graphical visualization [15].

2.17 Python for Embedded Communication

Python has become highly important in embedded communication systems because of its simplicity, portability, and extensive library ecosystem. Libraries such as PySerial simplify UART communication with embedded hardware, while asyncio and websockets libraries support asynchronous communication and real-time networking.

Python middleware applications can efficiently decode serial telemetry data, process JSON structures, and broadcast live information to multiple clients simultaneously. In the proposed project, Python is used for UART frame decoding, telemetry processing, asynchronous communication, and WebSocket server implementation [11][12][13].

2.18 Frontend Technologies Used

Frontend technologies such as HTML, CSS, and JavaScript are widely used for browser-based monitoring systems. HTML provides structural layout, CSS controls visual styling and animations, while JavaScript enables dynamic rendering, event handling, DOM manipulation, and WebSocket integration.

Modern browser dashboards support interactive visualization of real-time system activity through animations and live communication updates. JavaScript-based rendering allows smooth visualization of elevator movement, request indications, and emergency alerts.

2.19 Real-Time Monitoring Systems

Real-time monitoring systems are becoming increasingly important in industrial automation and intelligent infrastructure applications. Continuous monitoring improves operational visibility, fault diagnosis, and safety management by displaying live system behavior.

Telemetry-based visualization systems provide graphical representations of movement, requests, communication status, and event activity. Event-driven monitoring architectures improve responsiveness and simplify system analysis for operators.

The proposed project combines FPGA hardware control with telemetry transmission and browser-based visualization to create a unified real-time monitoring platform.

2.20 Literature Review

Several elevator controller implementations have been developed using relay logic, microcontrollers, and FPGA-based systems. Relay-based controllers suffer from wiring complexity, reduced scalability, and difficult maintenance. Microcontroller-based systems improved programmability but remained limited by sequential software execution under heavy concurrent operation.

FPGA-based elevator systems provide significant advantages including parallel processing, deterministic timing, reduced latency, scalability, and efficient handling of simultaneous requests [3]. Existing research primarily focuses on elevator movement control and floor scheduling; however, many implementations provide limited support for live telemetry and browser-based monitoring systems.

The proposed project extends traditional FPGA elevator systems by integrating FSM-based hardware control, UART telemetry, Python middleware, WebSocket communication, and browser-based real-time visualization into a unified hardware-software co-design platform. This integrated architecture demonstrates intelligent control, real-time embedded communication, live monitoring, and graphical visualization within a single FPGA-based automation system [8][16][17].

Chapter 3 : System Analysis and Requirements

3.1 Existing System Analysis

Elevator systems are widely used in residential buildings, commercial complexes, hospitals, and industrial infrastructures for vertical transportation. Traditional elevator controllers were primarily designed using relay-based circuits and electromechanical control systems. These systems were capable of basic floor movement and request servicing; however, they suffered from several limitations such as complex wiring, large hardware size, poor scalability, and difficult maintenance procedures. As the number of floors increased, the complexity of relay-based systems also increased significantly.

With the advancement of embedded systems, microcontroller-based elevator controllers became more common because they reduced hardware complexity and improved programmability. Microcontroller systems simplified movement control, floor scheduling, and user request handling through software implementation. Although these systems improved flexibility, they still relied on sequential instruction execution, which limited their ability to efficiently process multiple tasks simultaneously. Under heavy request loads, response time and real-time performance could degrade.

Another limitation observed in many conventional elevator systems is the absence of live monitoring and communication capability. Most systems operate as isolated embedded controllers without providing real-time visibility of elevator status, movement direction, active requests, or emergency conditions. Troubleshooting and debugging such systems become difficult because internal operational states are not easily accessible during runtime.

Modern intelligent automation systems require faster response, deterministic timing behavior, scalability, and remote monitoring capability. These requirements have encouraged the adoption of FPGA-based solutions for real-time control applications. FPGA devices provide parallel hardware execution, configurable architecture, low latency, and efficient state-based control, making them highly suitable for advanced elevator controller implementation.

3.2 Proposed System Analysis

The proposed project implements a smart elevator control system using FPGA technology and Verilog Hardware Description Language (HDL). The system is developed on the Digilent Basys3 FPGA development board and combines real-time hardware control with live software-based monitoring.

The elevator controller is designed using a Finite State Machine (FSM) architecture in which each operating condition of the elevator is represented as a dedicated state. These states include idle operation, upward movement, downward movement, door handling, and emergency operation. The FSM continuously monitors user requests and determines the movement direction and servicing sequence according to the active floor requests.

The proposed system supports three categories of requests:

- Cabin requests generated from inside the elevator
- Hall UP requests generated from floors below
- Hall DOWN requests generated from upper floors

To improve movement efficiency, the controller uses directional scheduling logic that continues serving requests in the current direction before reversing movement. This minimizes unnecessary traversal and reduces response delay.

A major feature of the proposed system is the integration of UART-based telemetry communication. Elevator state information is continuously transmitted from the FPGA hardware to a host computer through the onboard USB-UART interface. A Python middleware application receives and processes the serial data before forwarding it to a WebSocket server. A browser-based dashboard developed using HTML, CSS, and JavaScript visualizes elevator activity in real time.

The proposed architecture combines hardware control, serial communication, asynchronous software processing, and browser visualization into a unified embedded monitoring system. The modular implementation also improves maintainability, debugging capability, and scalability for future expansion.

3.3 Functional Requirements

The elevator control system must satisfy several functional requirements to ensure stable and correct operation.

3.3.1 Floor Request Handling

The system must be capable of receiving and processing floor requests generated from:

- Cabin buttons
- Hall UP buttons
- Hall DOWN buttons

All requests must be stored and serviced correctly without loss of information.

3.3.2 Elevator Movement Control

The controller must:

- Move upward when higher floor requests exist
- Move downward when lower floor requests exist
- Stop at requested floors
- Continue movement in the same direction until all requests in that direction are completed

3.3.3 Door Operation

The elevator door system must:

- Open automatically when the elevator reaches a requested floor
- Remain open for a predefined duration
- Close automatically before movement resumes

3.3.4 Emergency Handling

During emergency conditions:

- Normal elevator operation must be suspended
- The controller must move the cabin to the nearest safe floor
- The door must open automatically
- Visual indicators must display emergency status
- Operation must remain halted until reset

3.3.5 Display and Status Indication

The FPGA board must provide hardware-based visual feedback using:

- LEDs
- Seven-segment display

The display system must indicate:

- Current floor
- Elevator direction
- Door state
- Emergency condition

3.3.6 UART Communication

The FPGA hardware must continuously transmit telemetry data containing:

- Current floor information
- Movement direction
- Door state
- Emergency state
- Request status

3.3.7 Real-Time Monitoring

The browser-based monitoring system must:

- Display elevator movement in real time
- Visualize request indicators
- Update automatically through WebSocket communication
- Display emergency alerts and status information

3.4 Non-Functional Requirements

3.4.1 Real-Time Performance

The controller must provide immediate response to user requests and state transitions with minimal processing delay.

3.4.2 Reliability

The system should maintain stable operation without entering invalid states or incorrect scheduling conditions.

3.4.3 Scalability

The modular architecture should support future expansion such as additional floors, advanced scheduling algorithms, or IoT-based monitoring integration.

3.4.4 Low Communication Latency

UART communication and WebSocket updates should occur with minimal delay to ensure smooth real-time visualization.

3.4.5 Maintainability

The design should be modular and easy to debug, modify, and upgrade in future implementations.

3.5 Hardware Requirements

The proposed system requires the following hardware components for implementation and testing.

Hardware Component	Purpose
Digilent Basys3 FPGA Board	Main FPGA implementation platform
Slide Switches	Simulate elevator floor requests
Artix-7 FPGA	Executes elevator control logic
Push Buttons	Emergency and reset controls
LEDs	Hardware status indication
Seven-Segment Display	Floor and movement display
USB-UART Interface	FPGA-to-PC communication
Host Computer	Monitoring and dashboard execution

The Basys3 FPGA board provides sufficient logic resources, clock generation capability, and communication support for implementing the complete elevator control architecture.

3.6 Software Requirements

The following software tools and libraries are required for system development and monitoring.

Software Tool	Purpose
Xilinx Vivado Design Suite	FPGA synthesis and implementation
Verilog HDL	Hardware design language
Python	Middleware and communication processing
pySerial Library	UART serial communication
asyncio Framework	Asynchronous task handling
websockets Library	WebSocket server implementation

HTML/CSS/JavaScript	Browser-based monitoring dashboard
Visual Studio Code	Development environment

3.7 Feasibility Study

3.7.1 Technical Feasibility

The project is technically feasible because FPGA hardware provides sufficient processing speed, configurable logic resources, and real-time operation capability. The Basys3 board includes integrated peripherals such as switches, LEDs, displays, and UART communication support, simplifying implementation and testing.

3.7.2 Economic Feasibility

The system uses cost-effective educational hardware and open-source software tools. No specialized industrial equipment is required, making the project economically practical for academic and research environments.

3.7.3 Operational Feasibility

The proposed architecture provides reliable elevator operation along with user-friendly monitoring capability. Real-time visualization improves debugging, system observation, and operational analysis.

3.8 Requirement Specification Table

Requirement ID	Requirement Description	Priority
FR-1	Accept cabin floor requests	High
FR-2	Accept hall UP requests	high
FR-3	Accept hall DOWN requests	high
FR-4	Control elevator movement	high
FR-5	Handle door operations	high

FR-6	Implement emergency control	high
FR-7	Transmit UART telemetry	high
FR-8	Provide real-time dashboard updates	medium
NFR-1	Ensure real-time operation	high
NFR-2	Maintain system reliability	high
NFR-3	Support scalability	medium
NFR-4	Minimize communication latency	medium

Chapter 4 : System Design and Architecture

4.1 Overall System Architecture

The proposed smart elevator controller is designed using a layered architecture that combines FPGA-based real-time hardware control with software-based monitoring and visualization. The complete system consists of multiple interconnected modules responsible for elevator control, communication, telemetry processing, and browser visualization.

The FPGA hardware layer acts as the central controller of the system. User requests generated through switches and push buttons are processed by the elevator control logic implemented using Verilog HDL. The controller continuously monitors floor requests, determines elevator movement direction, handles door operation, and manages emergency conditions.

A UART communication module integrated inside the FPGA continuously transmits elevator telemetry data to the host computer through the onboard USB-UART interface available on the Basys3 FPGA board. The telemetry information includes current floor position, movement direction, door status, emergency state, and active requests.

The software layer consists of a Python-based middleware application responsible for serial communication and telemetry decoding. The Python application receives UART frames from the FPGA, extracts elevator state information, and forwards the processed data to a WebSocket server.

The browser visualization layer connects to the WebSocket server and displays real-time elevator activity through an interactive graphical dashboard. The monitoring interface visualizes floor movement, request indicators, movement direction, door state, and emergency alerts dynamically.

The layered design improves modularity, maintainability, debugging capability, and future scalability of the system.

4.2 High-Level Block Diagram

The complete elevator controller consists of several functional blocks working together to perform real-time elevator operation and monitoring.

The Request Processing Unit receives floor requests generated using cabin switches and hall request buttons. These requests are forwarded to the Elevator FSM

Controller, which determines the operational behavior of the elevator according to scheduling logic and current movement state.

The Floor Logic Unit continuously monitors elevator position and determines whether the elevator must stop at the current floor. The Door Control Logic manages door opening and closing operations during floor servicing.

The UART Transmission Unit converts elevator state information into serial telemetry frames and transmits them to the host computer. The Python middleware decodes the serial data and forwards the processed information to the browser dashboard through WebSocket communication.

The browser interface provides graphical visualization of elevator operation and continuously updates the elevator state in real time.

4.3 Data Flow Architecture

The proposed system follows a continuous real-time data flow architecture. User requests generated using FPGA input switches are processed by the elevator controller FSM. Based on active requests and current movement direction, the FSM updates elevator states such as movement direction, floor position, and door status.

The UART telemetry module packages the elevator state information into serial communication frames and transmits the data through the onboard USB-UART interface. The host computer receives the serial telemetry data through a virtual COM port.

The Python middleware continuously monitors incoming UART frames, validates synchronization bytes, extracts telemetry information, and converts the received data into structured JSON objects. The processed JSON data is then broadcast to all connected browser clients using a WebSocket server.

The browser dashboard receives these real-time updates and dynamically renders elevator movement, request status, door operation, and emergency conditions without requiring manual page refresh.

The complete communication chain operates continuously and provides smooth real-time system monitoring.

4.4 Elevator Operational Flow

The elevator controller operates according to a state-driven execution flow. Initially, the elevator remains in the idle state while continuously monitoring floor requests generated by users.

When a valid request is detected, the controller determines whether the requested floor lies above or below the current elevator position. If higher floor requests exist, the elevator enters upward movement mode. Similarly, lower floor requests activate downward movement mode.

During elevator traversal, the controller continuously checks whether the current floor matches any active cabin or hall request. When a matching request is detected, the elevator stops, opens the door for a predefined duration, clears the request, and resumes movement according to remaining requests.

The scheduling logic prioritizes servicing requests in the current movement direction before reversing direction. This directional persistence minimizes unnecessary movement and improves elevator efficiency.

If no pending requests remain, the controller returns to the idle state and waits for new requests.

During emergency conditions, the normal operational sequence is interrupted and emergency handling logic is executed immediately.

4.5 FSM Design of Elevator Controller

The proposed elevator controller is implemented using a Finite State Machine (FSM) architecture. FSM-based design provides deterministic control behavior and simplifies implementation of sequential elevator operations.

Each operational condition of the elevator is represented as a separate FSM state. The major states implemented in the controller include idle state, upward movement state, downward movement state, door operation state, and emergency state.

In the idle state, the elevator remains stationary while continuously monitoring incoming requests. When higher floor requests are detected, the controller transitions to the upward movement state. Similarly, lower floor requests activate the downward movement state.

The door operation state is activated whenever the elevator reaches a requested floor. During this state, elevator movement is temporarily suspended while the door remains open for passenger entry and exit.

The emergency state overrides all normal operating states. When emergency input is activated, normal movement logic is suspended, the elevator moves to the nearest safe floor, and the door is opened automatically.

The FSM continuously evaluates request conditions, floor position, and emergency signals to determine state transitions.

4.6 Request Scheduling Algorithm

The elevator controller uses a directional scheduling mechanism inspired by the SCAN scheduling algorithm commonly used in disk scheduling systems.

The scheduler combines:

- Cabin floor requests
- Hall UP requests
- Hall DOWN requests

The controller continuously identifies the highest requested floor and the lowest requested floor relative to the current elevator position. While moving upward, the elevator services all upward-direction requests sequentially before reversing direction. Similarly, downward traversal services all downward requests before changing direction.

The scheduling algorithm reduces unnecessary elevator movement, minimizes repeated direction switching, and improves passenger waiting time.

Request servicing logic also ensures that cabin requests and hall requests are processed efficiently without starvation of lower-priority requests.

The scheduling system allows smooth multi-request handling even when several requests are generated simultaneously.

4.7 Emergency Handling Design

Passenger safety is an important requirement in elevator systems. The proposed controller integrates a dedicated emergency handling mechanism inside the FSM architecture.

When emergency mode is activated, all normal elevator operations are suspended immediately. The controller overrides standard movement scheduling and activates emergency handling logic.

The elevator is directed toward the nearest safe floor and movement speed remains controlled during emergency traversal. Once the elevator reaches the target floor, the door opens automatically to allow safe passenger exit.

The emergency state remains active until the system receives a reset signal. During this condition, all new floor requests are ignored to prevent unsafe operation.

Visual indications on both the FPGA hardware and browser dashboard notify users about the emergency condition. Real-time telemetry transmission continues during

emergency operation, allowing external monitoring systems to track elevator status continuously.

The emergency handling mechanism improves operational safety and fault tolerance of the proposed system.

4.8 Door Control System Design

The door control system is synchronized with elevator floor servicing operations. Whenever the elevator reaches a requested floor, movement logic is temporarily paused and the controller enters the door operation state.

The door opens automatically and remains open for a predefined timing interval controlled using clock division logic inside the FPGA. During this interval, passengers are allowed to enter or exit the elevator.

After the timeout duration expires, the controller closes the door and resumes elevator movement according to pending requests and current scheduling direction.

The synchronization between movement logic and door operation prevents unsafe elevator motion while the door is open.

The door control mechanism also interacts with emergency logic to ensure safe passenger evacuation during abnormal conditions.

4.9 Clock Divider Design

The Basys3 FPGA board provides a high-frequency 100 MHz onboard clock signal. Since elevator movement, display operation, and UART communication require slower timing signals, clock divider circuits are implemented inside the FPGA design.

Separate divided clock signals are generated for:

- Elevator movement timing
- Seven-segment display multiplexing
- UART baud rate generation
- Door timing control

The clock divider module reduces the high-frequency system clock into lower-frequency signals suitable for real-time elevator operation.

Clock synchronization ensures stable FSM execution and prevents timing instability during communication and display operations.

The divided clock architecture also improves hardware reliability and predictable system behavior.

4.10 UART Telemetry Architecture

The UART telemetry system provides real-time communication between the FPGA hardware and the host computer.

The telemetry module continuously collects elevator state information including:

- Current floor position
- Elevator movement direction
- Door status
- Emergency state
- Cabin request data
- Hall request data

The collected information is organized into structured serial communication frames and transmitted through the onboard USB-UART interface available on the Basys3 FPGA board.

Frame synchronization logic is implemented to ensure reliable serial communication and prevent invalid data interpretation. The UART transmission system operates at a standard baud rate of 9600 bits per second.

Continuous telemetry transmission enables external monitoring systems to observe elevator behavior in real time.

4.11 UART FSM Design

The UART transmission module is implemented using a dedicated FSM architecture responsible for serial data sequencing.

The UART FSM consists of multiple operational states including:

- Idle state
- Start bit transmission state
- Data transmission state
- Stop bit generation state
- Cleanup state

During transmission, the UART controller serializes telemetry bytes and transmits them sequentially according to asynchronous communication timing requirements.

Baud rate timing is generated using clock division logic derived from the FPGA system clock. The UART FSM ensures proper synchronization between transmitted bits and receiver sampling intervals.

The FSM-based UART design improves communication reliability and simplifies serial transmission control.

4.12 Seven-Segment Display Design

The seven-segment display system provides local visualization of elevator state information directly on the FPGA board.

The display controller uses multiplexing techniques to drive multiple display digits using shared signal lines. The current floor number is continuously displayed on the seven-segment display during elevator operation.

Additional display patterns are used to indicate movement direction, idle condition, and system reset state.

Clock-based multiplexing enables smooth display rendering without visible flickering while minimizing FPGA I/O resource usage.

The display system assists in hardware debugging and provides immediate local feedback during FPGA testing.

4.13 LED Indication System

The onboard LEDs available on the Basys3 FPGA board are used for real-time hardware status indication.

The LED indication system displays:

- Active floor requests
- Elevator movement direction
- Door state
- Emergency condition
- Operational status

LED-based monitoring simplifies hardware-level debugging and allows rapid observation of elevator activity during testing.

The LED indicators operate synchronously with the FSM states and communication logic.

4.14 Web Dashboard Architecture

The browser-based monitoring dashboard provides graphical visualization of elevator activity using HTML, CSS, and JavaScript technologies.

The dashboard connects to the Python WebSocket server and continuously receives live elevator telemetry updates. The interface dynamically renders elevator movement, request indicators, movement direction, door status, and emergency alerts.

Animated elevator visualization is implemented to simulate real-time shaft movement and floor traversal. Cabin request indicators and hall request indicators update automatically according to active requests received from the FPGA hardware.

The dashboard also displays communication status and system connectivity information. Real-time updates occur asynchronously without requiring browser refresh operations.

The browser visualization layer significantly improves monitoring capability and provides a user-friendly interface for observing elevator operation.

4.15 Database and Data Storage Considerations

The proposed system primarily focuses on real-time monitoring and therefore does not require permanent database storage during normal operation.

Elevator telemetry data is processed dynamically and transmitted directly between the FPGA hardware, Python middleware, and browser dashboard. Temporary runtime data structures are used to maintain current elevator state information.

Although database integration is not implemented in the current design, the modular communication architecture supports future expansion for:

- Historical movement logging
- Fault analysis
- Predictive maintenance
- Usage statistics
- Cloud-based monitoring
- Remote IoT integration

The existing telemetry framework can be extended easily to support external database systems without major modification to the FPGA hardware design

Chapter 5 – Implementation and Software Interface

5.1 Introduction to Module-Wise Design

The proposed elevator controller is implemented using a modular design methodology in which each hardware and software component performs a dedicated function. Modular implementation improves readability, debugging capability, scalability, and maintainability of the overall system.

The FPGA design is divided into multiple Verilog modules responsible for elevator control, request scheduling, floor detection, UART communication, display handling, and emergency management. The software layer consists of Python middleware and a browser-based monitoring interface responsible for telemetry processing and real-time visualization.

The modular architecture allows independent testing and verification of each subsystem before complete system integration.

5.2 Top-Level Module: `lift_fpga_top.v`

The `lift_fpga_top.v` module acts as the top-level integration layer of the FPGA design. This module connects all hardware submodules and manages communication between them.

The top-level module receives:

- User input requests
- Clock signals
- Reset signals
- Emergency inputs

It coordinates interaction between:

- Elevator FSM controller
- Request processing module
- Floor logic module
- UART transmission module
- Display controller
- LED indication system

Clock division logic is also implemented inside the top-level architecture to generate slower timing signals required for elevator movement, display multiplexing, and UART communication.

The telemetry sequencer inside the top-level module continuously collects elevator state information and forwards it to the UART transmission unit for real-time serial communication.

The modular integration approach simplifies hardware organization and improves system scalability.

5.3 Elevator Core Module: **Lift8.v**

The **Lift8.v** module implements the core elevator control logic using a Finite State Machine architecture. This module acts as the primary decision-making unit of the elevator controller.

The module continuously monitors:

- Current floor position
- Cabin requests
- Hall UP requests
- Hall DOWN requests
- Movement direction
- Emergency state

Based on active requests, the FSM determines:

- Elevator movement direction
- Floor stopping conditions
- Door operation timing
- State transitions
- Emergency handling behavior

The FSM transitions between multiple operational states including idle, upward movement, downward movement, door operation, and emergency handling.

Directional scheduling logic ensures that requests in the current direction are serviced before reversing movement. This minimizes unnecessary traversal and improves operational efficiency.

The module also generates status signals required for LED indication, display output, and telemetry communication.

5.4 Floor Logic Module: **floor_logic.v**

The **floor_logic.v** module is responsible for floor matching and stop detection operations.

The module continuously compares:

- Current floor position
- Active cabin requests
- Active hall requests

Whenever the current floor matches a pending request, the module generates a stop signal for the elevator controller. The stop detection mechanism activates door opening logic and clears the serviced request from the request queue.

The floor logic module also assists in:

- Idle state generation
- Request completion detection
- Door trigger conditions
- Direction synchronization

The implementation improves modularity by separating floor comparison logic from the main FSM controller.

5.5 Request Logic Module: **request_logic.v**

The **request_logic.v** module manages request registration and scheduling assistance for the elevator controller.

The module stores:

- Cabin requests
- Hall UP requests
- Hall DOWN requests

Request arrays are continuously updated according to switch inputs generated by users. The module identifies:

- Maximum requested floor
- Minimum requested floor
- Pending request availability

The request management system assists the FSM in determining movement direction and scheduling decisions.

The modular request processing design simplifies multi-request handling and allows efficient scheduling implementation inside the FPGA architecture.

5.6 UART Transmission Module: `uart_tx.v`

The `uart_tx.v` module implements asynchronous serial communication between the FPGA hardware and the host computer.

The UART transmitter operates using an FSM-based architecture responsible for serial data transmission sequencing. The transmission process includes:

- Start bit generation
- Data bit transmission
- Stop bit generation
- Transmission completion handling

The UART module converts telemetry bytes into serial bit streams according to asynchronous communication standards.

A baud rate generator derived from the FPGA system clock produces the timing signal required for 9600 baud communication.

The module continuously transmits elevator telemetry information including:

- Current floor
- Direction state
- Door state
- Emergency status
- Request information

Reliable UART communication enables continuous real-time monitoring of elevator operation.

5.7 XDC Constraint File

The FPGA design uses an XDC constraint file for hardware pin mapping and clock configuration.

The constraint file defines:

- Input switch mappings
- Push button mappings
- LED output pins
- Seven-segment display connections
- UART TX pin assignment

- Clock signal configuration

Pin mapping ensures correct physical connection between FPGA logic signals and Basys3 hardware peripherals.

Clock constraints are also defined to maintain proper synthesis timing and stable hardware operation.

The XDC configuration is essential for successful FPGA implementation and hardware verification.

5.8 RTL Schematic Explanation

After synthesis in Xilinx Vivado, the FPGA design is converted into a Register Transfer Level (RTL) schematic representing the hardware structure of the implemented system.

The RTL schematic illustrates:

- Module hierarchy
- Signal flow connectivity
- FSM integration
- Clock routing
- Communication interfaces

The synthesized design confirms proper interconnection between:

- Elevator control logic
- Request processing modules
- UART transmitter
- Display controllers

RTL visualization assists in hardware verification and debugging by providing graphical representation of the implemented logic architecture.

5.9 Resource Utilization

The FPGA implementation utilizes configurable hardware resources available on the Artix-7 FPGA device present on the Basys3 development board.

Major FPGA resources used include:

- Lookup Tables (LUTs)
- Flip-flops
- Clock buffers

- Input/output pins
- Routing resources

The modular implementation minimizes unnecessary hardware utilization and maintains efficient logic organization.

The design comfortably fits within the available FPGA resource limits, allowing future expansion and additional feature integration.

5.10 Timing Analysis

Timing analysis is performed after synthesis and implementation to verify stable FPGA operation.

The timing verification process checks:

- Setup timing requirements
- Hold timing requirements
- Clock synchronization
- Signal propagation delays

Clock divider modules and UART baud rate generators are verified to ensure correct timing operation.

The analysis confirms that all hardware modules operate synchronously without timing violations under the configured FPGA clock frequency.

Proper timing verification improves system reliability and stable hardware execution.

5.11 FPGA Hardware Implementation

The proposed elevator controller is implemented and tested on the Digilent Basys3 FPGA development board containing the Xilinx Artix-7 FPGA device.

The FPGA board provides:

- Slide switches for request inputs
- Push buttons for emergency and reset operations
- LEDs for status indication
- Seven-segment display for floor visualization
- USB-UART communication interface

The hardware setup allows real-time testing of elevator movement, request scheduling, emergency handling, and telemetry communication.

The onboard USB-UART interface enables direct serial communication with the host computer without requiring additional external communication hardware.

The FPGA hardware implementation validates the practical operation of the proposed elevator controller architecture.

5.12 Python UART Bridge

A Python-based middleware application is developed to receive telemetry data transmitted by the FPGA through UART communication.

The Python application continuously monitors the serial COM port and receives incoming telemetry frames from the FPGA hardware.

The middleware performs:

- Serial data reception
- Frame synchronization
- Byte extraction
- State decoding
- JSON data conversion

The decoded telemetry information is converted into structured runtime data containing:

- Elevator floor position
- Movement direction
- Door state
- Emergency condition
- Active request arrays

The Python bridge acts as an intermediate communication layer between FPGA hardware and the browser visualization system.

5.13 UART Testing Utility

The UART communication system is verified using terminal-based telemetry monitoring.

The terminal utility displays:

- Raw UART frames
- Decoded floor information
- Movement direction
- Door status

- Cabin requests
- Hall requests

Continuous telemetry monitoring confirms successful real-time communication between FPGA hardware and the host computer.

The UART testing interface also assists in debugging frame synchronization and verifying telemetry integrity during runtime.

The testing results demonstrate stable asynchronous communication without data corruption or frame loss.

5.14 WebSocket Server Design

A WebSocket server is implemented inside the Python middleware to enable real-time communication between the backend processing layer and browser clients.

The WebSocket server continuously broadcasts live elevator telemetry updates received from the FPGA hardware.

The server manages:

- Client connections
- Real-time data streaming
- JSON transmission
- Connection synchronization

Asynchronous communication techniques are used to ensure smooth real-time updates without blocking serial communication tasks.

The WebSocket-based architecture enables efficient browser visualization and live system monitoring.

5.15 JSON State Structure

The processed telemetry information is converted into structured JSON objects before transmission to the browser interface.

The JSON structure contains:

- Current floor
- Movement direction
- Door state
- Emergency status
- Cabin request arrays

- Hall request arrays

The structured format simplifies browser-side parsing and visualization.

JSON serialization also improves interoperability and future scalability for external system integration.

5.16 Web-Based Monitoring Dashboard

A browser-based dashboard is developed using HTML, CSS, and JavaScript technologies to visualize elevator activity in real time.

The monitoring dashboard displays:

- Elevator shaft animation
- Current floor position
- Movement direction
- Door status
- Cabin requests
- Hall requests
- Emergency alerts

The dashboard receives live updates through WebSocket communication and dynamically updates interface components without requiring browser refresh operations.

Animated visualization improves user interaction and provides intuitive monitoring of elevator behavior.

The dashboard interface significantly enhances debugging capability and real-time system observation.

5.17 Frontend Design Components

The frontend interface is designed using multiple web technologies for responsive visualization and interactive monitoring.

HTML is used for structural organization of interface elements including:

- Floor indicators
- Request buttons
- Elevator shaft visualization
- Status panels

CSS is used for:

- Layout styling
- Animation effects
- Responsive design
- Visual appearance

JavaScript manages:

- WebSocket communication
- Dynamic rendering
- State synchronization
- Animation updates

The frontend design provides smooth real-time visualization and user-friendly interaction.

5.18 JavaScript Logic

The JavaScript layer acts as the client-side processing engine of the monitoring dashboard.

The JavaScript implementation handles:

- WebSocket client communication
- Incoming JSON parsing
- DOM manipulation
- Real-time state updates
- Elevator movement animation
- Emergency state rendering

The interface continuously updates visualization components according to incoming telemetry information received from the WebSocket server.

The asynchronous event-driven architecture ensures smooth interface operation without blocking browser rendering.

5.19 Real-Time Communication Flow

The complete communication chain operates continuously in real time.

The FPGA hardware generates telemetry data and transmits it using UART communication. The Python middleware receives and decodes the serial frames before forwarding the processed information through a WebSocket server.

The browser dashboard receives the telemetry updates and dynamically renders elevator movement and operational states.

The communication sequence consists of:

- FPGA → UART
- UART → Python Middleware
- Python → WebSocket Server
- WebSocket → Browser Dashboard

The complete architecture enables seamless hardware-software integration and real-time monitoring.

5.20 Advantages of the Monitoring System

The proposed monitoring architecture provides several advantages over conventional standalone elevator controllers.

Major advantages include:

- Real-time monitoring capability
- Live telemetry visualization
- Improved debugging support
- Hardware-software integration
- Low communication latency
- Scalable architecture
- User-friendly interface
- Efficient request monitoring
- Enhanced system observability

The modular communication framework also supports future expansion toward cloud monitoring, IoT integration, predictive maintenance, and remote diagnostics.

Chapter 6. Simulation and Experimental Results

6.1 Introduction to Verification

Verification and testing are important stages in FPGA-based digital system design. After implementation of the smart elevator controller, extensive testing was performed to validate the correctness, stability, and reliability of the hardware and software subsystems.

The testing process included:

- Functional verification of elevator movement
- Request scheduling validation
- Emergency condition testing
- UART communication testing
- Web dashboard synchronization testing
- Hardware implementation verification on FPGA

Both simulation-based verification and real-time hardware testing were performed to ensure proper system operation under different conditions.

The experimental validation confirmed that the elevator controller successfully handled multi-floor requests, real-time telemetry communication, and browser-based monitoring.

6.2 Simulation Environment

The FPGA design was developed and simulated using the Xilinx Vivado Design Suite. Simulation was performed before hardware implementation to verify FSM behavior, request scheduling logic, UART communication timing, and display operation.

The simulation environment included:

- Verilog HDL source modules
- Testbench modules
- Timing simulation setup
- FSM verification signals
- UART waveform observation

Testbench modules generated artificial floor requests and control signals to evaluate elevator response under multiple operating conditions.

Timing simulations verified proper synchronization between:

- Clock divider modules
- FSM transitions
- UART transmission timing
- Door control delays

Simulation-based verification reduced hardware debugging complexity and improved implementation reliability.

6.3 Functional Testing

Functional testing was performed to validate the operational behavior of the elevator controller under normal operating conditions.

The following operations were tested:

- Cabin request servicing
- Hall UP request handling
- Hall DOWN request handling
- Multi-request scheduling
- Direction switching
- Idle state handling

During testing, requests were generated using FPGA slide switches and push buttons. The elevator successfully responded to all valid requests and serviced floors according to the implemented scheduling algorithm.

The FSM correctly maintained directional persistence by servicing all requests in the current movement direction before reversing direction. This reduced unnecessary movement and improved traversal efficiency.

The controller also returned to the idle state automatically after completion of all pending requests.

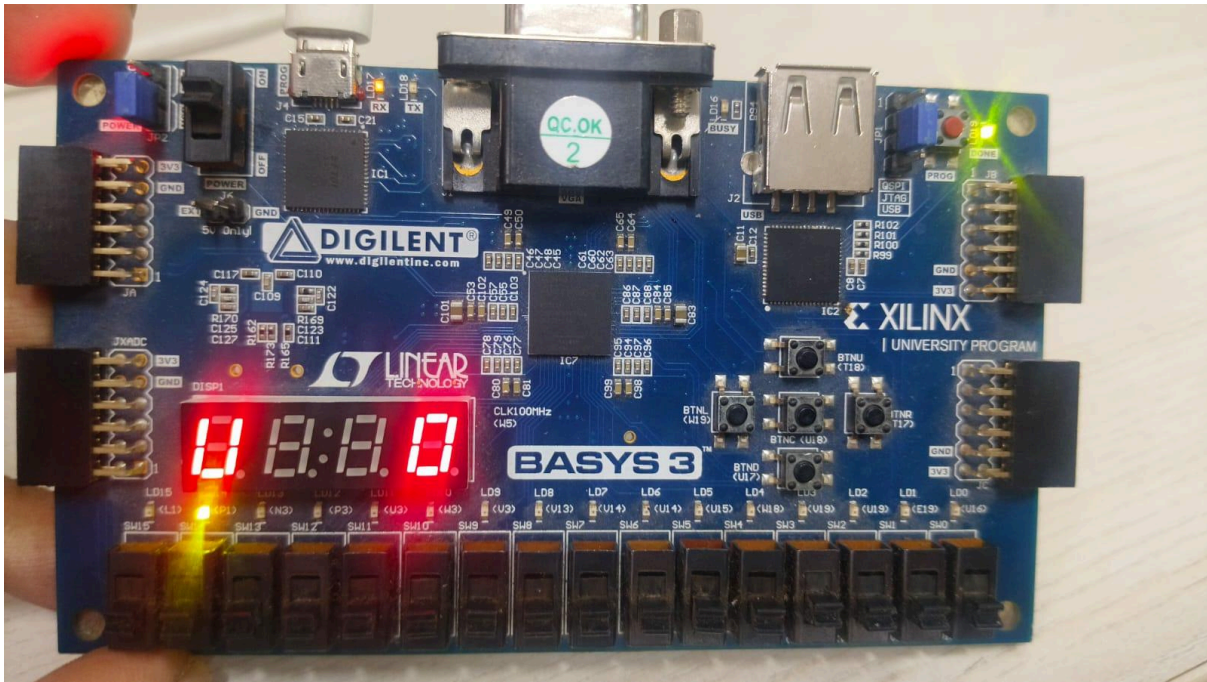


Figure 6.1: Real-Time Elevator Operation on Basys3 FPGA Board

6.4 Emergency Condition Testing

Emergency handling verification was performed to validate passenger safety mechanisms implemented inside the FSM controller.

During testing, the emergency input was activated while the elevator was moving between floors. The controller immediately suspended normal scheduling operation and initiated emergency handling behavior.

The following conditions were verified:

- Emergency stop activation
- Nearest floor detection
- Safe stopping operation
- Door opening during emergency
- Request clearing behavior
- Emergency state indication

The system successfully redirected the elevator to the nearest safe floor and automatically opened the door after stopping. All new requests were ignored while the emergency condition remained active.

Emergency telemetry updates were continuously transmitted to the monitoring dashboard during testing.

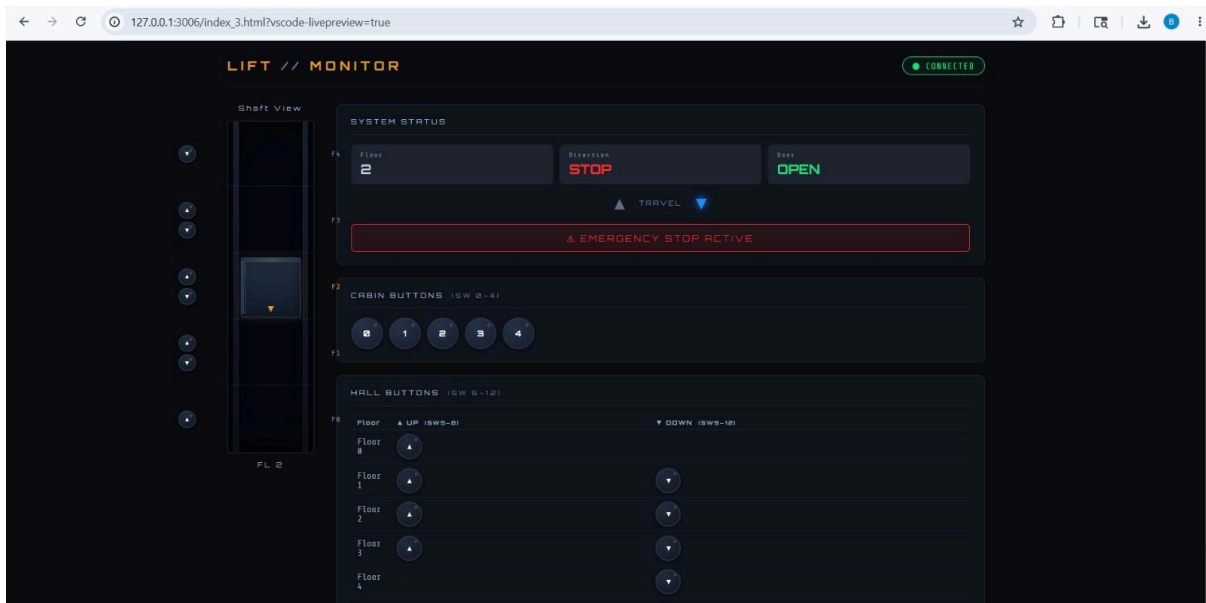


Figure 6.2: Emergency State Visualization on Monitoring Dashboard

6.5 UART Communication Testing

UART communication testing was performed to verify reliable telemetry transmission between the FPGA hardware and the host computer.

The UART transmitter continuously generated serial telemetry frames containing:

- Current floor number
- Elevator direction
- Door state
- Cabin requests
- Hall requests
- Emergency status

The Python middleware successfully received and decoded incoming UART frames through the onboard USB-UART interface.

The following communication parameters were validated:

- Baud rate synchronization
- Frame alignment
- Byte integrity
- Continuous streaming stability
- Frame decoding accuracy

Testing confirmed stable asynchronous communication at 9600 baud without communication failure or frame corruption.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[Frame #0075] Raw: 90 10 01 00 | Floor=0 UP Door=CLOS Cabin=[0, 0, 0, 0, 1] HallUp=[0, 0, 0, 0]
HallDn=[0, 0, 0, 0]
[Frame #0076] Raw: 90 10 01 00 | Floor=0 UP Door=CLOS Cabin=[0, 0, 0, 0, 1] HallUp=[0, 0, 0, 0]
HallDn=[0, 0, 0, 0]
[Frame #0077] Raw: 90 10 01 00 | Floor=0 UP Door=CLOS Cabin=[0, 0, 0, 0, 1] HallUp=[0, 0, 0, 0]
HallDn=[0, 0, 0, 0]
[Frame #0078] Raw: 90 10 01 00 | Floor=0 UP Door=CLOS Cabin=[0, 0, 0, 0, 1] HallUp=[0, 0, 0, 0]
HallDn=[0, 0, 0, 0]
[Frame #0079] Raw: 90 10 01 00 | Floor=0 UP Door=CLOS Cabin=[0, 0, 0, 0, 1] HallUp=[0, 0, 0, 0]
HallDn=[0, 0, 0, 0]
[Frame #0080] Raw: 90 10 01 00 | Floor=0 UP Door=CLOS Cabin=[0, 0, 0, 0, 1] HallUp=[0, 0, 0, 0]
HallDn=[0, 0, 0, 0]
[Frame #0081] Raw: 90 10 01 00 | Floor=0 UP Door=CLOS Cabin=[0, 0, 0, 0, 1] HallUp=[0, 0, 0, 0]
HallDn=[0, 0, 0, 0]
[Frame #0082] Raw: 90 00 01 00 | Floor=0 UP Door=CLOS Cabin=[0, 0, 0, 0, 0] HallUp=[0, 0, 0, 0]
HallDn=[0, 0, 0, 0]
[Frame #0083] Raw: 90 00 01 00 | Floor=0 UP Door=CLOS Cabin=[0, 0, 0, 0, 0] HallUp=[0, 0, 0, 0]
HallDn=[0, 0, 0, 0]
[Frame #0084] Raw: 90 00 01 00 | Floor=0 UP Door=CLOS Cabin=[0, 0, 0, 0, 0] HallUp=[0, 0, 0, 0]
HallDn=[0, 0, 0, 0]
[Frame #0085] Raw: 90 00 01 00 | Floor=0 UP Door=CLOS Cabin=[0, 0, 0, 0, 0] HallUp=[0, 0, 0, 0]
HallDn=[0, 0, 0, 0]

```

Figure 6.3: UART Telemetry Output and Frame Decoding

6.6 WebSocket Communication Testing

WebSocket testing was performed to verify real-time communication between the Python middleware and the browser dashboard.

The WebSocket server continuously streamed telemetry updates received from the FPGA hardware. Browser clients successfully received live elevator state updates without requiring page refresh operations.

The following parameters were verified:

- Client connection establishment
- Real-time data broadcasting
- JSON transmission accuracy
- Browser synchronization
- Continuous update stability

The browser dashboard remained synchronized with FPGA hardware operation throughout testing.

Real-time updates occurred with minimal latency, enabling smooth monitoring of elevator movement and request activity.

6.7 Display Testing

The seven-segment display and LED indication systems were tested during FPGA implementation.

The seven-segment display successfully displayed:

- Current floor number
- Idle condition
- Directional movement state

LED indicators correctly represented:

- Active requests
- Direction status
- Emergency state
- Operational status

Display multiplexing logic operated without visible flickering during continuous operation.

Hardware display testing improved debugging capability and validated physical FPGA output behavior.

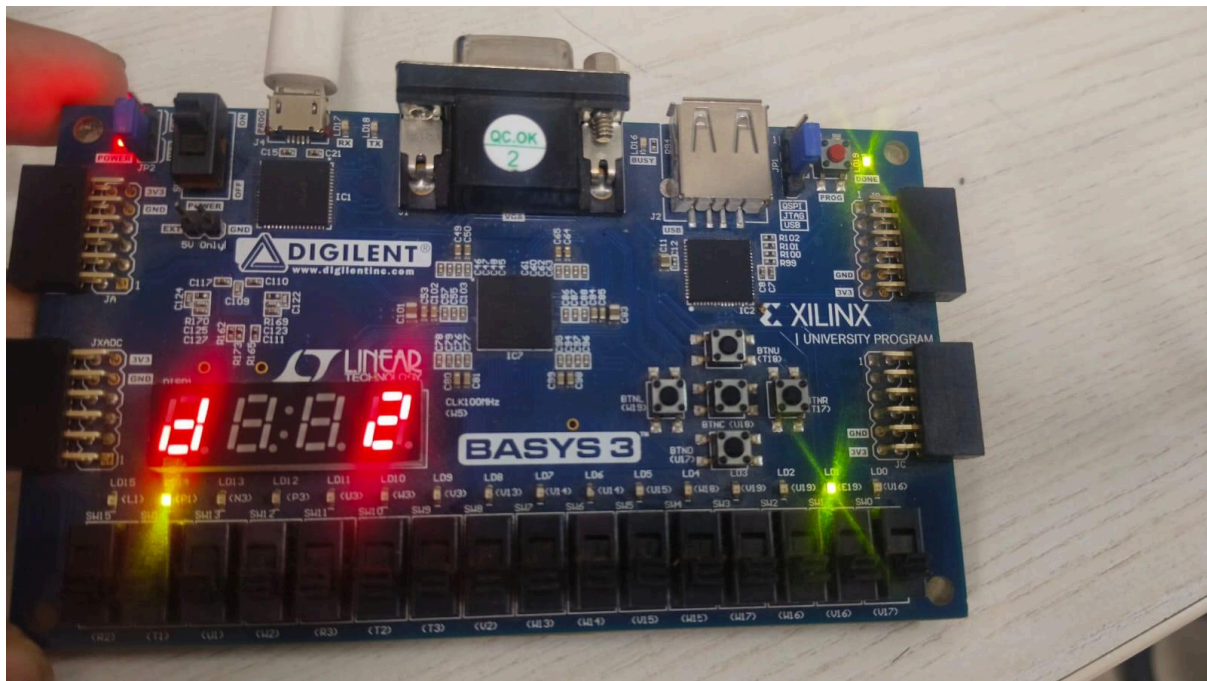


Figure 6.4: Seven-Segment Display and LED Status Indicators

6.8 Hardware Testing on FPGA

After simulation verification, the complete design was synthesized and programmed onto the Digilent Basys3 FPGA development board.

Real-time hardware testing validated:

- Elevator FSM operation
- Request scheduling

- Direction switching
- UART communication
- Emergency handling
- Browser monitoring

The onboard switches and push buttons were used to generate floor requests and emergency conditions.

The FPGA hardware successfully executed all elevator control operations in real time without instability or timing violations.

The testing confirmed correct interaction between:

- Hardware control logic
- UART telemetry system
- Python middleware
- WebSocket communication
- Browser visualization layer

6.9 Test Cases and Results

Multiple test cases were executed to evaluate system functionality and reliability.

Test Case		Input Condition		Expected Result		Actual Result	Status
Cabin Test	Request	Floor 2 selected		Elevator moves to floor 2		Successful	Pass
Hall UP Request		UP request generated		Elevator moves upward		Successful	Pass
Hall Request	DOWN	DOWN generated	request	Elevator moves downward		Successful	Pass
Multi-Request Handling		Multiple active	requests	Sequential servicing		Successful	Pass
Emergency Activation		Emergency enabled	input	Elevator safely stops		Successful	Pass

UART Communication	Telemetry transmission enabled	Frames correctly received	Successful	Pass
Dashboard Monitoring	Browser connected	Real-time updates visible	Successful	Pass

The experimental results confirmed stable and reliable system operation under different testing conditions.

6.10 Waveform Analysis

Waveform analysis was performed using the Vivado simulator to verify internal hardware signals and FSM timing behavior.

The following signals were analyzed:

- Elevator movement states
- Door control timing
- UART transmission signals
- FSM state transitions
- Clock divider outputs

Simulation waveforms confirmed correct sequential execution of FSM transitions and stable UART transmission timing.

No timing instability or invalid state transitions were observed during waveform analysis.

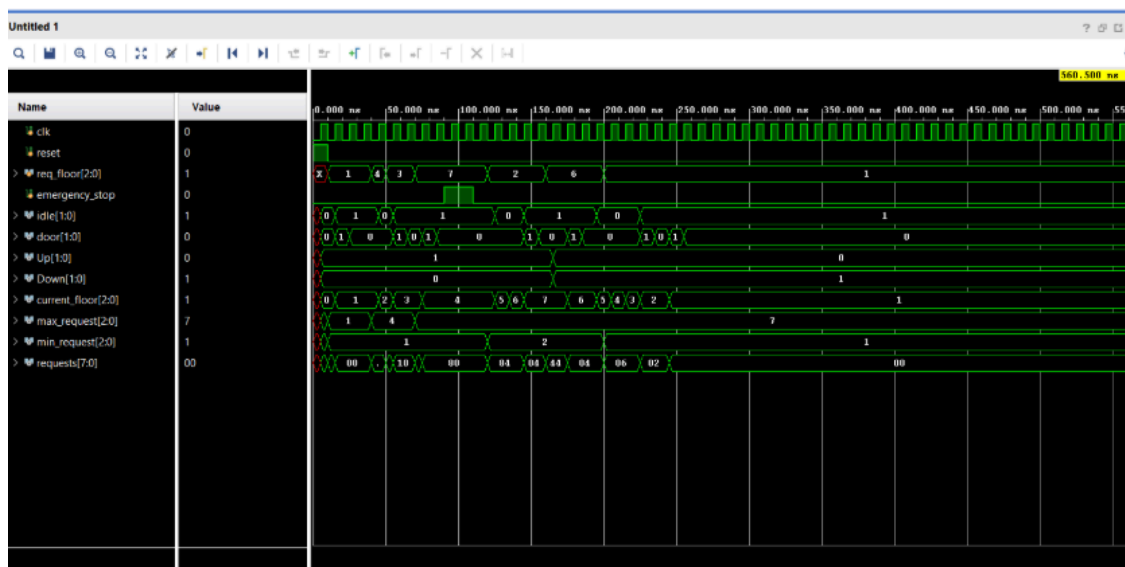


fig 6.5 waveform

6.11 Performance Analysis

The proposed FPGA-based elevator controller demonstrated stable real-time performance during testing.

Performance evaluation included:

- Response time analysis
- UART communication throughput
- Real-time monitoring latency
- Hardware resource utilization

The FPGA implementation provided deterministic hardware execution with fast response to incoming requests.

UART telemetry operated continuously without frame loss, while the browser dashboard maintained near real-time synchronization with FPGA hardware.

The modular design also maintained efficient utilization of FPGA logic resources.

6.12 Error Analysis

During development and testing, several implementation challenges were encountered and resolved.

The major issues included:

- UART synchronization mismatch
- Timing instability during early FSM testing
- Display multiplexing synchronization
- WebSocket update delays
- Serial frame alignment issues

These issues were resolved through:

- Clock divider optimization
- Improved UART frame synchronization
- FSM state refinement
- Debugging using terminal telemetry output
- Continuous hardware verification

The final implementation operated reliably after optimization and debugging.

6.13 Predictive Maintenance Feature

An additional advanced feature implemented in the proposed system is predictive maintenance and fault monitoring support. Along with normal elevator control and real-time monitoring, the system continuously observes elevator operational behavior to identify abnormal conditions and possible maintenance requirements.

The telemetry architecture developed in this project allows continuous collection of operational data such as:

- Elevator movement activity
- Request frequency
- Door operation behavior
- Emergency activations
- Communication status
- System runtime patterns

This real-time operational information can be analyzed to detect unusual system behavior that may indicate developing faults or hardware stress conditions. For example, repeated emergency activations, irregular movement behavior, excessive request load, or communication interruptions can be treated as indicators of potential maintenance requirements.

The browser dashboard also provides continuous visibility of elevator operational states, allowing early detection of abnormal conditions by operators. Since telemetry data is transmitted continuously through UART and WebSocket communication, the system supports future expansion toward intelligent fault prediction and automated maintenance alerts.

The implemented predictive maintenance approach improves:

- System reliability
- Operational safety
- Fault detection capability
- Maintenance planning
- Monitoring efficiency

Unlike conventional elevator controllers that only react after faults occur, the proposed architecture supports proactive monitoring by continuously observing system behavior in real time.

Although the current implementation performs basic predictive monitoring through telemetry observation and fault-state analysis, the modular design can be extended further using:

- Machine learning algorithms

- Cloud-based analytics
- Historical data logging
- IoT sensor integration
- Predictive fault classification

The inclusion of predictive maintenance functionality demonstrates the scalability and modern applicability of the FPGA-based smart elevator monitoring architecture.

6.14 Experimental Results and Discussion

Experimental testing demonstrated successful integration of FPGA hardware control and real-time monitoring systems.

The implemented controller successfully achieved:

- Multi-floor elevator operation
- Real-time request scheduling
- Direction optimization
- Emergency handling
- UART telemetry transmission
- Browser-based visualization

The FPGA implementation provided deterministic and parallel hardware execution compared to traditional software-only approaches.

The browser dashboard significantly improved monitoring capability and provided intuitive visualization of elevator operation.

The system demonstrated the effectiveness of combining:

- FPGA-based digital design
- Embedded communication
- Python middleware processing
- WebSocket streaming
- Real-time frontend visualization

The proposed architecture can be extended further for:

- Multi-elevator systems
- IoT-based monitoring
- Cloud telemetry
- Smart building automation system

Chapter 7 – Conclusion and Future Works

7.1 Conclusion

The project successfully demonstrated the design and implementation of a smart FPGA-based elevator control system using Verilog Hardware Description Language and the Digilent Basys3 FPGA development board. The developed system combined real-time hardware control, UART-based telemetry communication, Python middleware processing, and browser-based visualization into a unified intelligent monitoring platform.

The elevator controller was implemented using a Finite State Machine (FSM) architecture capable of handling multiple operational states such as idle condition, upward movement, downward movement, door control, and emergency handling. The system efficiently processed cabin requests and hall requests while maintaining optimized traversal direction using a scheduling mechanism inspired by the SCAN algorithm.

Hardware implementation on the FPGA board verified stable and deterministic real-time operation. The controller successfully performed:

- Multi-floor request handling
- Direction-based scheduling
- Door operation control
- Emergency condition handling
- Seven-segment display visualization
- LED-based monitoring
- UART telemetry transmission

The integration of UART communication enabled continuous transmission of elevator telemetry data from the FPGA hardware to a host computer. A Python-based middleware application decoded the serial frames and forwarded real-time elevator status updates through a WebSocket server to the browser dashboard.

The browser-based monitoring system provided live graphical visualization of:

- Elevator movement
- Current floor
- Door status
- Request indicators
- Emergency conditions
- Operational states

One of the important achievements of the project was the successful integration of predictive maintenance concepts through continuous telemetry monitoring and fault observation. The implemented architecture demonstrated how real-time system data can be used for proactive monitoring and future intelligent maintenance systems.

The project also highlighted the advantages of FPGA technology in automation and control applications. Compared to traditional sequential software systems, FPGA-based implementation provided:

- Parallel hardware execution
- Deterministic timing behavior
- Faster response time
- Stable communication performance
- Reliable real-time processing

Extensive simulation and experimental testing confirmed correct operation of all major system components including FSM transitions, UART communication, scheduling logic, emergency handling, and browser synchronization.

Overall, the project successfully achieved all intended objectives and demonstrated the practical use of FPGA technology for smart elevator automation and real-time embedded monitoring systems.

7.2 Future Works

Although the current implementation successfully demonstrates a functional smart elevator controller, several advanced features and improvements can be added in future versions to further enhance system intelligence, scalability, and industrial applicability.

7.2.1 Increasing Number of Floors

The current system is implemented and tested for a limited number of floors on FPGA hardware. Future versions can support larger multi-floor buildings by expanding request arrays, display logic, and scheduling mechanisms.

Scalable FPGA resource utilization makes it possible to extend the architecture for high-rise elevator systems.

7.2.2 Multi-Elevator Coordination System

Future work can include implementation of multiple elevators operating within the same building environment.

A centralized scheduling controller can distribute requests dynamically among elevators based on:

- Elevator position
- Current load
- Direction of movement
- Waiting time optimization

Multi-elevator coordination would significantly improve passenger efficiency in large buildings.

7.2.3 AI-Based Elevator Scheduling

Artificial Intelligence and machine learning algorithms can be integrated into the scheduling system to improve elevator efficiency.

AI-based scheduling could:

- Predict user movement patterns
- Optimize elevator traversal
- Reduce waiting time
- Improve energy efficiency
- Learn peak traffic behavior

Intelligent scheduling would enable adaptive and self-optimizing elevator operation.

7.2.4 IoT Integration

The telemetry architecture can be expanded into a complete Internet of Things (IoT) monitoring system.

Future IoT integration may include:

- Remote cloud monitoring
- Mobile notifications
- Wireless telemetry
- Real-time fault alerts
- Smart building integration

IoT connectivity would allow remote monitoring and maintenance from any location.

7.2.5 Cloud-Based Monitoring

The current browser dashboard operates locally using WebSocket communication. Future systems can integrate cloud servers for centralized elevator monitoring and analytics.

Cloud integration would support:

- Long-term telemetry storage
- Historical data analysis
- Predictive maintenance analytics
- Remote debugging
- Multi-building monitoring

This would improve scalability and industrial deployment capability.

7.2.6 Mobile Application Integration

A dedicated Android or iOS application can be developed for remote elevator monitoring and control.

The mobile application may provide:

- Live elevator status
- Maintenance alerts
- Emergency notifications
- Fault monitoring
- Remote diagnostics

Mobile integration would improve accessibility and operational convenience.

7.2.7 Voice Command Support

Future elevator systems can integrate voice recognition modules for hands-free operation.

Voice command functionality could allow users to:

- Select floors
- Request emergency assistance
- Receive audio notifications
- Interact with accessibility features

This enhancement would improve user experience and accessibility support.

7.2.8 Advanced Sensor Integration

Additional hardware sensors can be integrated to improve safety and intelligent automation.

Possible sensors include:

- Weight sensors
- Infrared door sensors
- Motion sensors
- Obstacle detection sensors
- Vibration monitoring sensors

These sensors would improve passenger safety and support predictive fault detection.

7.2.9 Energy Optimization Techniques

Future versions may include energy-efficient movement algorithms and power optimization techniques.

Possible improvements include:

- Intelligent idle positioning
- Load-aware scheduling
- Regenerative braking simulation
- Low-power operating modes

Energy optimization would reduce operational cost and improve sustainability.

7.2.10 Advanced Safety Systems

Additional safety mechanisms can be integrated for industrial-grade deployment.

Future safety features may include:

- Fire emergency handling
- Overload detection
- Power failure recovery
- Automatic rescue operation
- Backup communication systems

These improvements would increase system reliability and passenger safety during abnormal conditions.

References

- [1] Digilent Inc., *Basys 3 FPGA Board Reference Manual*, 2016. Available: https://www.amd.com/content/dam/amd/en/documents/university/aup-boards/XUPBasys3/documentation/Basys3_rm_8_22_2014.pdf
- [2] AMD/Xilinx, *Vivado Design Suite User Guide (UG893)*. Available: <https://docs.amd.com/r/en-US/ug893-vivado-ide>
- [3] Chu, Pong P., *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*, Wiley, 2008. Available: <https://www.wiley.com/en-us/FPGA+Prototyping+by+Verilog+Examples%3A+Xilinx+Spartan-3+Version-p-9780470185322>
- [4] Palnitkar, Samir, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Prentice Hall, 2003. Available: <https://www.amazon.com/Verilog-HDL-Guide-Digital-Synthesis/dp/0130449113>
- [5] IEEE, *IEEE Standard for Verilog Hardware Description Language (IEEE Std 1364-2005)*, 2006. Available: <https://standards.ieee.org/ieee/1364/3641/>
- [6] Texas Instruments, *UART Communication Protocol Overview*. Available: <https://www.ti.com/lit/an/slyt113/slyt113.pdf>
- [7] Navabi, Zainalabedin, *Verilog Digital System Design*, 2nd Edition, McGraw-Hill, 2010.
- [8] FPGA Elevator Controller Research Paper, *FPGA Based Elevator Controller*. Available: https://www.researchgate.net/publication/283041078_FPGA_Based_Elevator_Controller
- [9] Fette, I., and Melnikov, A., *The WebSocket Protocol (RFC 6455)*, IETF, 2011. Available: <https://datatracker.ietf.org/doc/html/rfc6455>
- [10] Chart.js Documentation, 2024. Available: <https://www.chartjs.org/docs/latest/>

- [11] Python Software Foundation, *pySerial Documentation*. Available: <https://pyserial.readthedocs.io/>
- [12] Python Software Foundation, *asyncio – Asynchronous I/O Documentation*. Available: <https://docs.python.org/3/library/asyncio.html>
- [13] websockets Library Documentation. Available: <https://websockets.readthedocs.io/>
- [14] Digilent Inc., *Basys 3 FPGA Master XDC and Resource Center*. Available: <https://digilent.com/reference/programmable-logic/basys-3/start>
- [15] Mozilla Developer Network (MDN), *WebSocket API Documentation*. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- [16] Fatima Kashif, *Designing a Simple Elevator System Using FSM on FPGA*. Available: <https://share.google/mAz1dZG4jtdGXOZY4>
- [17] FPGA Elevator Controller Video Tutorial, YouTube. Available: <https://youtu.be/VtbPexCocVo?si=dZwznwXWLiiBGpHV>